

在 LLM 推理框架 (如 vLLM、SGLang 等) 中, `--max-num-seqs` 代表的是**“最大并发序列数”**。用大白话来说, 它其实就是模型在同一时间最多能同时处理的请求数量 (也就是我们常说的 **Batch Size** 的硬性上限)。

为了让你更好地理解它的作用, 我们可以把它放在你之前提供的两张幻灯片的上下文中来拆解:

1. 它的核心定义 (餐厅的“桌子数量”)

如果把 GPU 处理推理请求比作一家餐厅营业:

- `--max-model-len` (最大序列长度): 代表餐厅里每张桌子最多能坐多少人 (单个请求的上下文有多长)。
- `--max-num-seqs` (最大并发序列数): 代表餐厅里一共有多少张桌子。如果你设置为 256, 意味着系统最多同时接待 256 个用户的独立对话请求。哪怕此时显存还有空余, 只要并发数达到了 256, 第 257 个请求就得排队等待。

2. 它与 `--max-num-batched-tokens` 的区别 (经常被混淆)

在倒数第二张图的“内存与 KV Cache”表格中, 这两个参数紧挨着:

- `--max-num-seqs` 控制的是“请求的个数” (比如: 同时处理 100 句话)。
 - `--max-num-batched-tokens` 控制的是“Token 的总吞吐量” (比如: 这 100 句话加起来, 单次计算的总字数不能超过 8192 个 Token)。
- 系统在实际组装 Batch 的时候, 必须同时满足这两个上限条件。

3. 为什么上一张图建议“增大 `--max-num-seqs`”?

这与你上一问中关于“GPU 算力闲置”的问题直接相关。

在上一张图的右侧“如何解决?”中, 第一条明确写着: “增大 `--max-num-seqs`, 直到 GPU Compute 达 85%+”。

- 背景: 当 GPU 计算单元利用率低 (<50%) 时, 说明工人没吃饱。
- 动作: 此时我们去调大 `--max-num-seqs` (比如从默认的 256 调到 512)。
- 结果: 系统被允许同时接纳更多的用户请求 (Batch Size 变大了)。并行的矩阵乘法任务成倍增加, 原本闲置的计算单元 (CUDA Cores) 瞬间被海量的计算任务填满, GPU 的计算利用率就会从 50% 飙升到 85% 以上。

总结来说:

`--max-num-seqs` 是你用来压榨 GPU 算力、提升系统整体吞吐量 (Throughput) 的最直接的油门踏板。只要你的显存 (VRAM) 还没爆, 就应该尽可能地调大这个参数, 让 GPU 满负荷为你工作。